

# Algoritmos-de-Espera-Ocupada.pdf



**JesusLagares**



**Programación Concurrente y de Tiempo Real**



**3º Grado en Ingeniería Informática**



**Escuela Superior de Ingeniería  
Universidad de Cádiz**



[Accede al documento original](#)

antes



**Descarga sin publi  
con 1 coin**



Después

**WUOLAH**



Importante

Puedo eliminar la publi de este documento con 1 coin

¿Cómo consigo coins? → Plan Turbo: barato  
→ Planes pro: más coins

pierdo espacio



# Algoritmos de Espera Ocupada



Los algoritmos de **espera ocupada** son las primeras soluciones históricas al problema de la **exclusión mutua**. En lugar de bloquear al proceso, lo mantienen **girando activamente** (busy waiting) mientras espera su turno para entrar en la sección crítica.

## Explicación

Estos algoritmos:

- NO usan primitivas de bloqueo del sistema operativo (como `wait()`, `sleep()`, semáforos...).
- Se basan únicamente en **variables compartidas** y **lecturas/escrituras repetidas**.
- Hacen que el proceso espere comprobando continuamente si puede avanzar → **consumen CPU de forma constante**.
- Requieren memoria compartida → **no funcionan en sistemas distribuidos**.
- Son difíciles de diseñar, probar y transportar entre arquitecturas (cachés, niveles de memoria, reordenamientos...). Además de muy ineficientes.

Por eso hoy se estudian solo como **historia de la concurrencia**, pero siguen siendo muy preguntables en examen.

Tenemos:

## Exclusión Mutua para 2 Procesos

### Dekker

Fue la **primera solución correcta** de exclusión mutua sin soporte del hardware.

Características:

- Usa **banderas** indicando intención de entrar.
- Usa un **turno** para romper empates.
- Requiere "**cortesía**": si ambos quieren entrar, uno cede voluntariamente.

► Dekker pasó por varias versiones (1-4 incorrectas) hasta llegar a la versión 5 (correcta).

Para el examen habría que saberse los problemas que presenta cada intento, ya que suelen poner los códigos de sus intentos como códigos a examinar y decir si tienen problemas de seguridad, vivacidad, etc.

## Peterson

Características:

- Mucho más simple que Dekker.
- Usa:
  - un **turno**,
  - un array `flag[i]` indicando "quiero entrar".
- También emplea cortesía.

Importante:

- Funciona solo para **2 procesos**.
- Es correcto siempre que se garantice *atomocidad en lectura/escritura de las variables*.

## Hyman

Otro intento temprano de exclusión mutua para 2 procesos.

- Pretendía ser más simple que Dekker.
- Finalmente se demostró **incorrecto** (permite violación de exclusión mutua).

En examen puede aparecer como:

**"Hyman es una solución incorrecta" → TRUE**

# Exclusión mutua para N procesos

## Eisenberg-McGuire

Algoritmo para **N procesos** inspirado en Dijkstra.

Características:

- Usa "cortesía" generalizada.
- Usa un **turno rotatorio**.
- Implementa tres estados por proceso:
  - QUIET (fuera)
  - WAITING
  - ACTIVE

Más complejo que Peterson, pero correcto para N procesos.

## Lamport

El algoritmo clásico para N procesos.

Muy conocido por la analogía:

Como en una panadería, cada proceso coge un número y espera a que le toque.

Características:

- Cada proceso elige un número mayor que todos los visibles.
- Gana el que tenga el número más pequeño (y en caso de empate, el ID menor).
- Garantiza:
  - exclusión mutua,
  - espera acotada (no hay inanición),
  - corrección para N procesos.

Importante

Puedo eliminar la publi de este documento con 1 coin

¿Cómo consigo coins? → Plan Turbo: barato  
→ Planes pro: más coins

pierdo  
espacio



Necesito  
concentración

ali ali ooh  
esto con 1 coin me  
lo quito yo...

WUOLAH



Extremadamente famoso porque fue uno de los primeros algoritmos que no necesitaban instrucciones especiales del hardware.